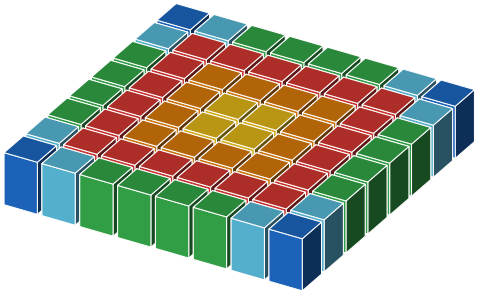
The 16GB Ceiling

Test-Time Training for ARC-AGI-2 on One Consumer GPU

Running the 2025 ARC Prize winning recipe, per-task test-time training
with augment-infer-reverse-vote selection, on one 16 GB card



16.28%

on a single 16 GB GPU

up from 12.79%, 13× faster

ARC Prize 2026 · Paper Track

ARC Prize 2026 Paper Track entry

Open source under MIT-0

Abstract

We run the 2025 ARC Prize winning recipe, a small grid model with per-task test-time training (TTT) and augment-infer-reverse-vote (AIRV) selection, on a single 16 GB consumer GPU, and ask what actually binds at that scale. The binding constraint is generation throughput, not model quality and not memory. Moving decode off the Hugging Face eager path onto llama.cpp buys a 13× speedup, turns an overnight failure-prone evaluation into a sub-90-minute one, and is what makes per-task TTT tractable at all, by hot-swapping each task's adapter on the fast engine. Building on the NVARC Qwen3-4B grid model, the system scores **12.79%** without TTT on the 120-task public evaluation set. The lift from TTT depends on regularization. A naive adapter overfits and ties the baseline, while a regularized, scale-tuned configuration reaches **17.6%** on the 84 trainable tasks and lifts the full evaluation to a blended **16.28%**. On the hidden main-track leaderboard the same recipe scores **13.61%** with TTT and 6.53% without, and the public-to-hidden transfer is itself a finding. The no-TTT baseline halves, and across the follow-on submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score, so public-eval gains must be re-validated in the exact deployed condition before they are trusted. We give an honest accounting of what the recipe costs and where the effort should go for anyone working with one consumer GPU rather than the winner's hardware. The contest itself provides four L4 GPUs, 96 GB together, where the offline frontier already passes 50% through refinement and program synthesis, so we read the 16 GB result as the efficient floor of a larger climb rather than a ceiling on the task. All code, the conversion recipe, and a development log of every measurement are open source under MIT-0.

Contents

1 Introduction **2**

1.1 Efficient abstraction is the point 2

1.2 Contributions 3

2 Related Work **4**

3 System and Method **4**

3.1 Model and prompting 4

3.2 The engine is the bottleneck 5

3.3 AIRV: augment, infer, reverse, vote 6

3.4 Per-task test-time training 7

4 Results **8**

4.1 Two regimes, and the leaderboard progression 8

4.2 The engine speedup 9

4.3 Test-time training 10

4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall 11

5 Discussion **12**

5.1 Four lessons that generalize 12

5.2 Accuracy per GPU-hour 13

5.3 Limitations and ongoing work 13

5.4 Scaling to the full contest hardware 15

5.5 A routed dual-arm system 15

6 Conclusion **17**

A Reproducibility **17**

B Extended results **18**

C Engineering log of walls and dead ends **19**

D Prompt and serialization format **20**

E Data and model provenance **20**

Reader's guide. Section 1 motivates efficient ARC and states the contributions. Section 2 places the work against the 2025 winning systems. Section 3 is the method, covering the model and prompt format (3.1), the engine bottleneck and its fix (3.2), the AIRV selection rule (3.3), and the per-task TTT bridge (3.4). Section 4 reports the numbers. Section 5 draws the lessons. Readers who want to reproduce the system should go straight to Appendix A. The per-scale tables and significance detail are in Appendix B, the honest list of dead ends in Appendix C, the exact prompt and 16-token vocabulary in Appendix D, and data licensing and BibTeX in Appendix E.

1 Introduction

1.1 Efficient abstraction is the point

ARC-AGI-2 [1, 2] is a benchmark of abstraction and reasoning. Each task gives a few input/output grid pairs that demonstrate a transformation, and the solver must apply that transformation to a held-out input. The tasks are designed to be easy for humans and hard for machines, and the competition is run *offline*, which rules out frontier API models and forces every entrant to ship a self-contained system. The 2025 ARC Prize was won by NVARC at 24.0% on ARC-AGI-2 using a 4B grid model with per-task TTT [3, 4]. The same recipe reproduced across the top teams. They fine-tune a small model on augmented grid data, train a fresh low-rank adapter on each test task, then sample many augmented candidates and vote.

The recipe is effective but front-loaded with compute. The base models are pre-trained on clusters of H100 or GH200 GPUs, though the winning offline submission itself runs on four L4 GPUs. Most researchers do

not have that hardware, and the ARC Prize's stated mission is *efficient* skill acquisition, not skill bought with compute. We therefore characterize the recipe under a constraint we think is widely shared, a single RTX 5070 Ti with 16 GB, on the Blackwell architecture, running native Windows. The aim on one card is not to top the leaderboard but to build a working, reproducible system, measure honest numbers, and map where the small model stops being enough. That map is the efficient floor of a larger climb. The same contest hands every entrant four L4 GPUs, 96 GB together, and the offline frontier on that hardware has already passed 50% with refinement loops and program synthesis [3]. We read the 16 GB walls not as the limit of the task but as the exact places scaling up has to break, and Section 5.4 lays out that path. Figure 1 shows a representative task our system solves end to end.

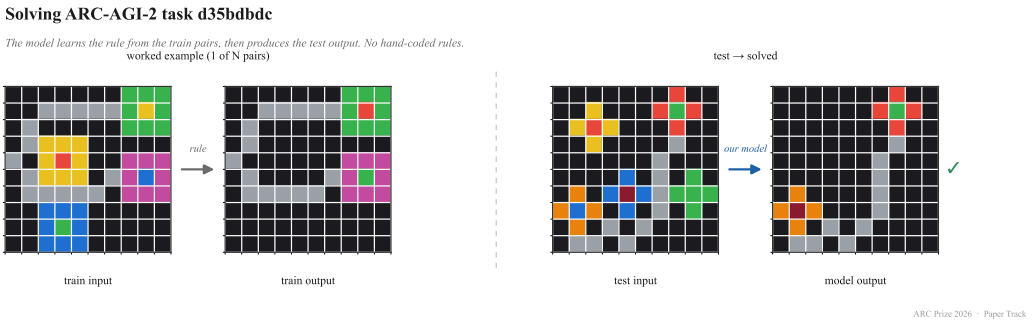


Figure 1: A real ARC-AGI-2 evaluation task solved by the no-TTT pipeline, shown as colour grids. The few-shot train pairs establish the rule, and the model produces the held-out test output exactly. Grids are rendered with the model's own 16-colour serialization (Appendix D).

1.2 Contributions

1. A reproducible offline ARC-AGI-2 system for the custom-vocabulary NVARC model on a single 16 GB card, with a clean **12.79%** no-TTT baseline (Section 4, Appendix A).
2. The finding that, on this hardware class, the inference *engine* dominates every other knob, since moving decode off the Hugging Face eager path onto llama.cpp is worth a 13× speedup (Section 3.2).
3. A practical bridge that makes per-task TTT runnable on one card by hot-swapping GGUF adapters on the fast engine, and a measured, regularization-dependent lift to a **16.28%** blend (Section 3.4, 4.3).
4. An "accuracy per GPU-hour" accounting that names the three walls a compute-poor practitioner hits first, the decode ceiling, the VRAM limit on large-grid TTT, and adapter overfitting (Section 5).
5. The measured finding that public-eval gains do not transfer to the hidden set. The no-TTT baseline halves from 12.79% to 6.53%, and across seven leaderboard submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score when deployed (Section 4, 5.1).
6. A reading of why those walls are a property of a small model on one card rather than of the task, and a concrete path that scales the same verifier, TTT, and selection onto the full 96 GB the contest provides, where the offline frontier already passes 50% (Section 5.4).

7. A complete open-source release, the conversion patches, the pipeline, the TTT harness, and a development log of every measurement and dead end behind these numbers (Appendix C).

2 Related Work

ARC and the measure of intelligence. ARC was introduced to measure skill-acquisition efficiency rather than skill itself [1]. ARC-AGI-2 [2] raises compositional difficulty while keeping tasks human-solvable, widening the human/machine gap that frontier models had begun to close on ARC-AGI-1.

Test-time training. The dominant 2025 approach fits a small per-task adapter at inference, trading test-time compute for generalization. The ARC Prize 2025 technical report [3] identifies this, together with heavy synthetic pre-training and disciplined engineering, as the core of the winning NVARC solution [4], whose released model we build on directly. We hold this recipe fixed and ask what survives a single consumer GPU.

Augmentation and voting. Predicting under the eight dihedral symmetries of the grid, inverting each prediction back to the original frame, and voting is a standard self-consistency device for ARC. It is cheap, training-free, and provides a usable confidence signal (Section 3.3).

Synthetic data. Procedural generators such as RE-ARC [6] and LLM-mutated corpora supply the augmented grid data these models are trained on. We use RE-ARC only in a base-model probe (Section 5.3), not in the results below.

Efficient and on-device inference. Our engineering contribution, getting a custom-vocabulary grid transformer to decode near the memory-bandwidth roofline on a consumer card and then serving per-task adapters on it, sits in the practical lineage of quantized GGUF inference. The novelty is not the engine but the demonstration that the full TTT recipe, adapter serving included, fits one 16 GB card, and a measurement of exactly where it stops.

3 System and Method

3.1 Model and prompting

We use the NVARC `qwen3_4b_grids15_sft139` model, a Qwen3-4B [5] with a 16-token vocabulary, the ten grid colours as digits, a newline, two role markers, and three specials (Appendix D). A grid is serialized one token per cell, and a colour value equals its token id. Because the role and special tokens cannot be produced by tokenizing text, we assemble every prompt at the token-id level as a ChatML few-shot sequence. Getting this byte-exact is the single most important correctness condition, since a one-token drift between training and inference silently destroys accuracy.

3.2 The engine is the bottleneck

Our most consequential finding concerns throughput. Decoding a 4B model is memory-bound. Each token streams the full weight matrix from VRAM, so for a weight footprint M on a card of bandwidth B the decode rate is bounded by a first-order roofline

$$r_{\max} = \frac{B}{M}. \tag{1}$$

With $B = 896$ GB/s, Eq. (1) gives 132 tok/s in bf16 ($M = 6.77$ GB), 249 in Q8_0 ($M = 3.6$ GB), and 444 in Q4_K_M ($M = 2.02$ GB). The standard Hugging Face eager path prefills healthily ($\approx$ 3,400 tok/s) but *decodes* at only $\approx$ 13 tok/s, about a tenth of the bf16 roof, because of per-token framework overhead with no CUDA graphs. The usual remedy, compiling the decode step, is unavailable on this platform. `torch.compile` needs Triton, and Triton has no native-Windows backend (Appendix C).

The fix is to change engines. We convert the model to GGUF and serve it with llama.cpp. The custom vocabulary needs two small, documented converter patches (Appendix A): the unknown tokenizer hash is mapped to the `qwen2` pre-tokenizer it actually uses, and two throwaway byte-pair merges are added to satisfy a loader that requires a non-empty merge list (they never fire on grid text). The prebuilt CUDA 12.4 binary runs on the Blackwell card through PTX just-in-time compilation. Measured decode rises to 102 tok/s in bf16, 170 in Q8_0, and 244 in Q4_K_M (Figure 2), a 13$\times$ speedup over eager at Q8_0, which we adopt. It is near-lossless (14 of 15 training tasks reproduced exactly under greedy decoding) and has a 3.6 GB footprint. The server's default prompt cache needs one note. It reserves $\approx$ 680 MiB per unique prompt, and since every ARC prompt is unique it yields no reuse and destabilizes sustained generation, so we disable it and run a single supervised stream.

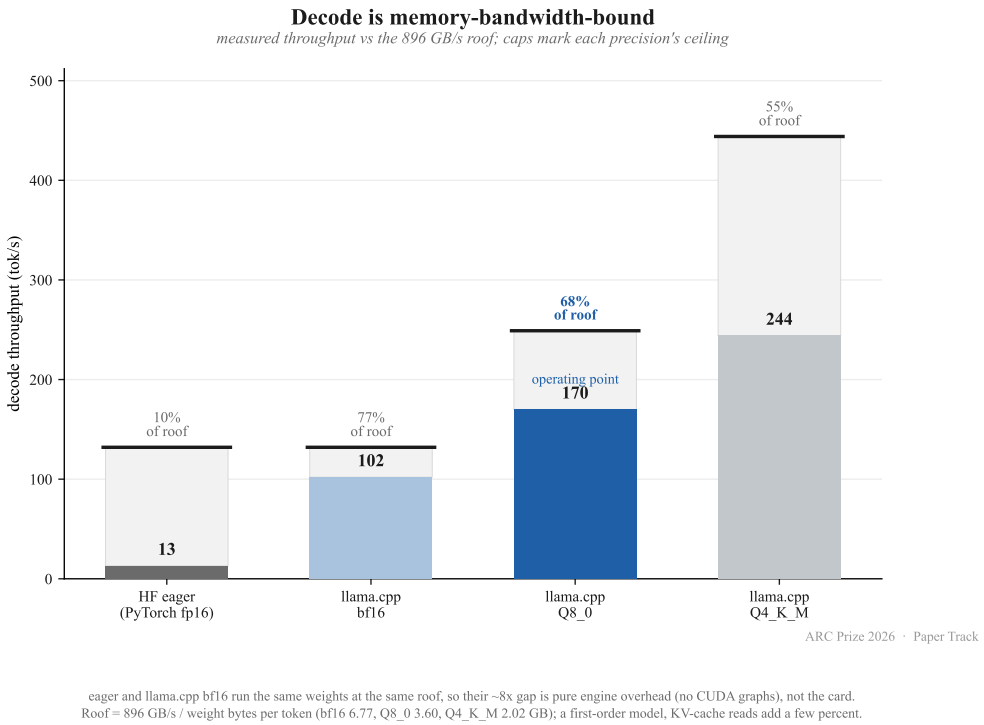


Figure 2: Decode throughput versus the memory-bandwidth roofline of Eq. (1) (the ceiling is a first-order model, bytes/token $\approx$ weight bytes). Bars are *measured* tok/s; the shaded gap to each cap is framework overhead, not the card. The eager path fills only $\approx 10\%$ of its roof, and moving decode to llama.cpp at Q8_0 recovers the $13\times$.

3.3 AIRV: augment, infer, reverse, vote

Let f be the served model and $\mathscr{G}$ the eight dihedral (D4) symmetries of the grid. For a test input x we generate one prediction per symmetry, invert it back to the original frame, and vote. Writing n_c for a candidate's vote count and $\bar{\ell}_c$ for its mean sequence log-prob,

$$c_g = g^{-1}\big(f(g(x))\big), \qquad \hat{Y} = \operatorname*{select}_{c}\,\big(n_c,\,\bar{\ell}_c\big), \qquad n_c = \big|\{g \in \mathscr{G} : c_g = c\}\big|, \quad \bar{\ell}_c = \frac{1}{n_c}\sum_{g:\,c_g=c} \log p(c_g), \tag{2}$$

where select returns the two distinct candidates with the highest vote count, ties broken by mean log-prob, and emits them as the two attempts the metric allows. The vote count is itself a usable confidence signal. Agreement across symmetries predicts correctness (Figure 6). The whole offline system (the trainer, the converter, and the server, which never share VRAM) is sketched in Figure 3.

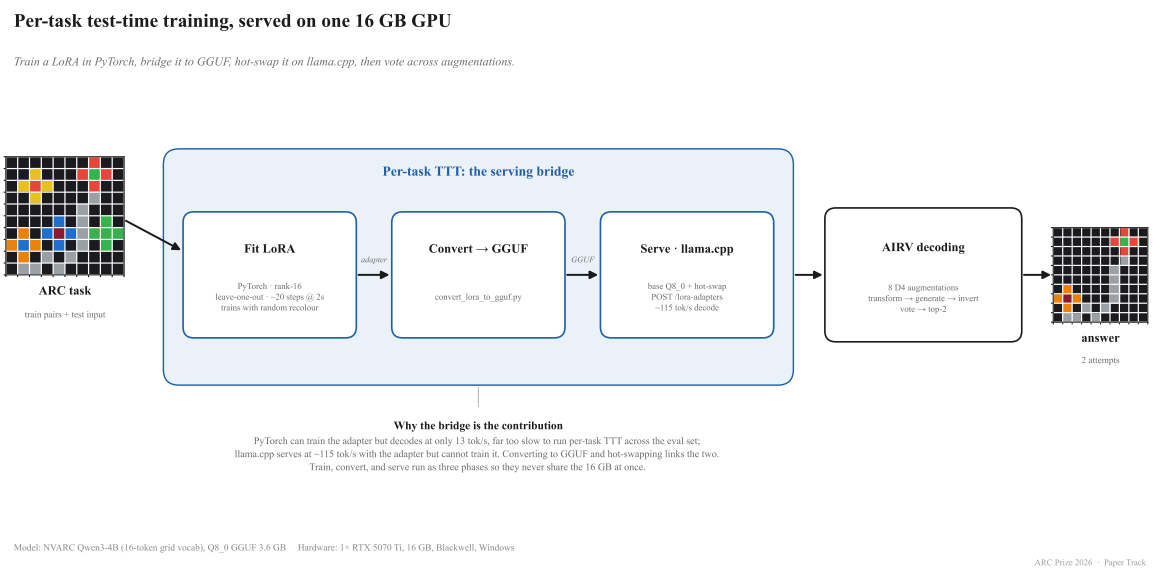


Figure 3: The offline system. The PyTorch trainer and the llama.cpp server never share VRAM; per-task adapters cross the bridge as GGUF and are hot-swapped on the fast engine per request.

3.4 Per-task test-time training

For TTT we fit a per-task LoRA adapter $\Delta W = sBA$ of rank r and scale s on the task's own train pairs T, with a leave-one-out target and the loss masked to the answer tokens:

$$\mathscr{L}(\theta) \;=\; -\sum_{(u,v)\in T}\;\sum_{t}\log p_\theta\big(v_t \mid v_{<t},\, \mathrm{ctx}_{T\setminus\{(u,v)\}},\, u\big), \tag{3}$$

drawing a fresh random augmentation each step. Capping the few-shot context to $\approx 1{,}600$ tokens drops a training step from ≈ 13 s to ≈ 2 s, so training is no longer the expensive part.

The harder problem is *serving* the trained adapter. TTT must run in PyTorch, where decoding is the slow 13 tok/s and intermittently becomes quadratic after a step. The fast engine does not train. We bridge them (Figure 3): train in PyTorch, save each adapter, convert it to a GGUF adapter, and hot-swap it on llama.cpp through its `/lora-adapters` endpoint, where decode holds ≈ 115 tok/s with no stalls. We run three phases (train all, convert all from a clean process, then serve and generate all) so the two frameworks never share VRAM (Figure 4). Large-grid tasks remain too VRAM-bound to train and are excluded by their largest-train-pair token length (Figure 4).

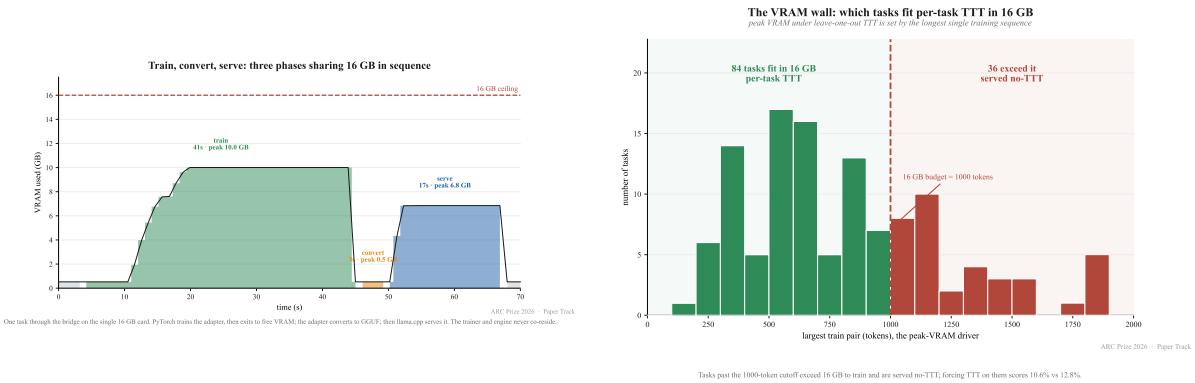


Figure 4: The left panel shows the three-phase schedule keeping PyTorch and llama.cpp from sharing the 16 GB at once, with the VRAM trace sampled live and the peaks measured. The right panel shows the VRAM wall, with each task placed by grid size and context length, and tasks above the operational cutoff excluded from TTT. In the right panel only the per-task points and the cutoff are measured. The translucent VRAM surface is a calibrated model, not a measurement, drawn only to make the wall legible, and the cutoff metric is the largest train pair.

4 Results

4.1 Two regimes, and the leaderboard progression

Two hardware regimes appear in this paper, and every number belongs to exactly one of them. Every public-eval number, the 12.79% baseline, the 17.6% trainable-task peak, and the 16.28% blend, is measured on the single RTX 5070 Ti against the 120-task public evaluation set, and all method development ran there. Every leaderboard number is measured by Kaggle's offline scoring rerun on the contest's four-L4 node, against the hidden 240-task set (120 semi-private plus 120 private). The submissions use the four L4s only to shard tasks for throughput, since the bf16 per-task training does not fit the 12-hour budget on one card. No single task needs more than one 16 GB GPU, so the method under test is the single-card recipe throughout.

The AIRV pipeline without TTT scores **12.79%** (22 of 172 test outputs) on the public evaluation, a clean single-stream run completing in ≈ 78 minutes, and **6.53%** on the hidden set, a halving we return to in Sections 5.1 and 5.3. Adding per-task TTT roughly doubles the hidden score. Table 1 gives the full progression.

Table 1: Leaderboard submissions, scored on the hidden 240-task set on the four-L4 node.

version	configuration	hidden score (%)
v1	no-TTT AIRV, eight greedy D4 views	6.53
v2	+ per-task TTT (160 tasks) + colour-perm	11.94
v2.1	scaled TTT (232 tasks), single scale, no colour-perm	**13.61**
v3	+ three-scale vote	error
v3.1	+ two-scale vote + colour-perm	12.78
v4	+ more-candidates sampling (four per view)	13.19
v5	+ white-box plausibility filter	12.78

The scaled single-scale configuration (v2.1) reached **13.61%**, ranking 468 of about 4000, and four later submissions failed to beat it. One errored in the scored rerun, and each of the three that scored carried exactly one lever that had lifted or tied the public evaluation, multi-scale voting with colour permutation, more-candidates sampling, and the white-box plausibility filter, and each regressed the hidden score (Section 4.4 gives the levers and the mechanism). The eight greedy D4 views at a single adapter scale are a local optimum that the candidate and voting side does not improve, and the repeated public-to-hidden reversal is itself a finding (Section 5.1). For reference the 2025 winner reached 24.0% with TTT [3] on the same Qwen3-4B base lineage with a more developed recipe.

4.2 The engine speedup

The 13$\times$ decode improvement from changing engines (Section 3.2, Figure 2) is the single most important result for this hardware class. It turned an overnight, frequently-failing process into a sub-90-minute evaluation. On one consumer GPU, the inference engine dominated every other knob we touched (Figure 5).

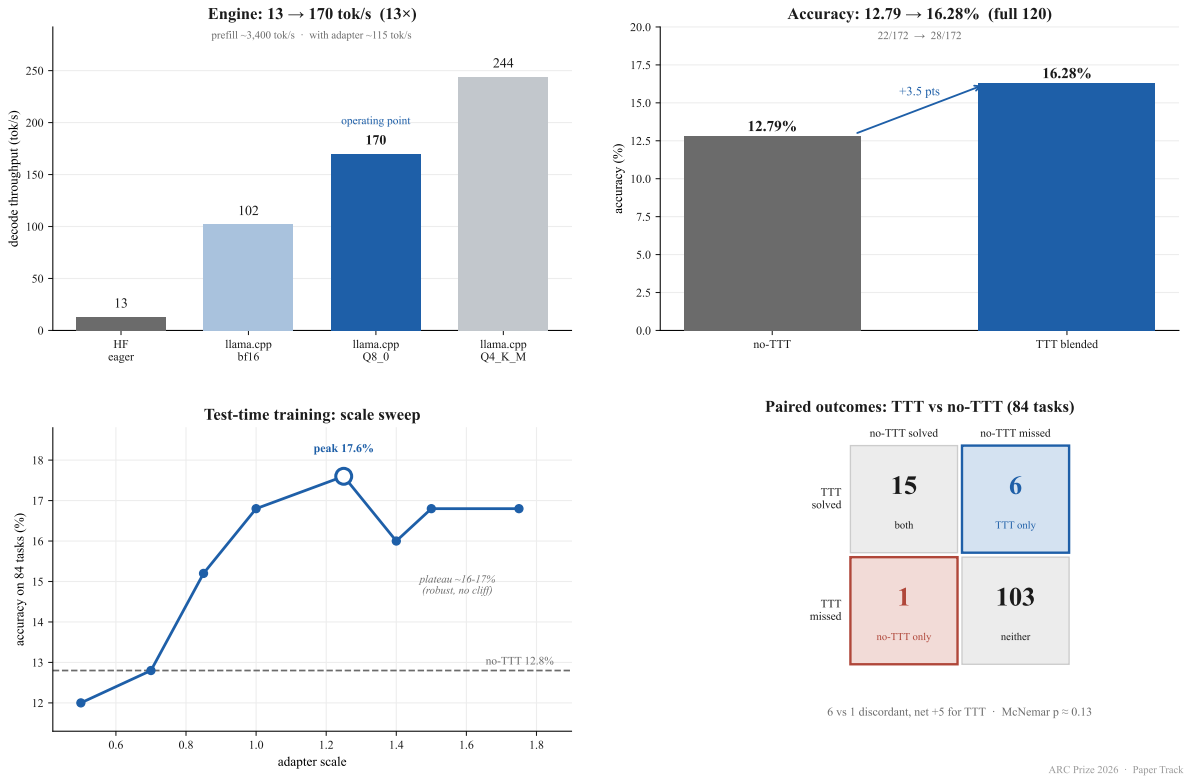


Figure 5: Engine choice, accuracy, and the adapter-scale curve. Accuracy peaks at scale 1.25 on the 84 trainable tasks, then declines, a real optimum rather than a monotone gain. All values are measured.

4.3 Test-time training

With generation fixed, we first swept the adapter configuration on 15 small-context tasks, scoring each against no-TTT on the same tasks (Table 2). The standard configuration overfits (its training loss reaches 0.000) and ties the baseline. The regularized configuration recovers a one-task lift. An over-aggressive reduction undertrains and ties again. That is coherent with an overfit diagnosis, though one task on 15 is only a hint.

Table 2: Adapter-configuration sweep on 15 small-context tasks (23 test outputs).

configuration	TTT	no-TTT
40 steps, rank 32, no dropout	3/23	3/23
20 steps, rank 16, dropout 0.1	**4/23**	3/23
8 steps, rank 8, dropout 0.05	3/23	3/23

We then validated the regularized configuration at scale, on the 84 evaluation tasks small enough to train on a 16 GB card, against no-TTT on the same tasks (Table 3). The lift holds and strengthens, adding $+5$ test outputs for a four-point gain ($\approx 31\%$ relative). The paired breakdown is clean. Of the tasks the

two methods disagree on, TTT is right on six that no-TTT misses and wrong on only one. With seven disagreements this is not individually significant (exact McNemar $p \approx 0.13$, Appendix B), but three measurements now agree in direction (sweep $+1$, scale $+5$, six-to-one discordant), and no-TTT scores 12.8% here against its 12.79% full baseline, so the subset is representative.

Table 3: Regularized TTT vs. no-TTT on the 84 trainable tasks (125 test outputs), scale 1.0.

method	correct	accuracy
regularized TTT	**21/125**	**16.8%**
no-TTT	16/125	12.8%

4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall

Five further levers, each measured on the 84 trainable tasks or the full evaluation (full per-scale numbers in Appendix B). The adapter *scale* matters. Accuracy peaks at 17.6% at scale 1.25, then declines (Figures 5, 6). Colour permutation lifts the no-TTT baseline to 14.4% but does *not* stack with the already colour-robust adapter. Forcing TTT onto the big-grid tasks does not help. The VRAM exclusion is a genuine ceiling, not a missed opportunity. The fourth lever is how many candidates the vote sees, and it cuts both ways. Sampling each of the eight D4 views four times raises the no-TTT score from 12.79% to 13.95% on the full evaluation, but like colour permutation it does not stack with the per-task adapter. Carried into a TTT submission those extra sampled candidates regressed the hidden score from 13.61% to 13.19%, and stacking multi-scale voting and colour permutation regressed it further to 12.78%. The adapter already concentrates the model on its single best answer, so more candidates help only the unsure no-TTT base and dilute the confident TTT one.

The fifth lever is a white-box plausibility filter adapted from a small-model recipe [7], which drops candidates that violate the task's colour palette or the train pairs' size pattern before the vote. On the public evaluation it is exactly neutral. The eight greedy D4 candidates are already plausible, so filtering changes no attempt on any of the 120 tasks, and where its inferred size rule misfires, on tasks whose train pairs coincidentally share one output size while the true rule varies, it empties the candidate list and the pipeline falls back to the unfiltered vote. Submitted as v5, it regressed the hidden score from 13.61% to 12.78%, a roughly two-task loss consistent with that size-rule tail landing on hidden tasks where the filter kept a plausible wrong candidate instead of emptying, though run-to-run TTT variance cannot be excluded from outside. The asymmetry is the lesson. A lever that measures neutral on the public set has no upside and only tail risk on the hidden one.

The eight greedy D4 views at a single scale are a local optimum that neither more votes nor vote filtering beats. The best blend, TTT at scale 1.25 where trainable and no-TTT elsewhere, reaches **16.28%** on the full 120-task evaluation, up from 12.79%. The honest summary is that naive per-task TTT *ties* a strong no-TTT AIRV baseline, while a regularized, scale-tuned TTT *beats* it by a consistent if modest margin.

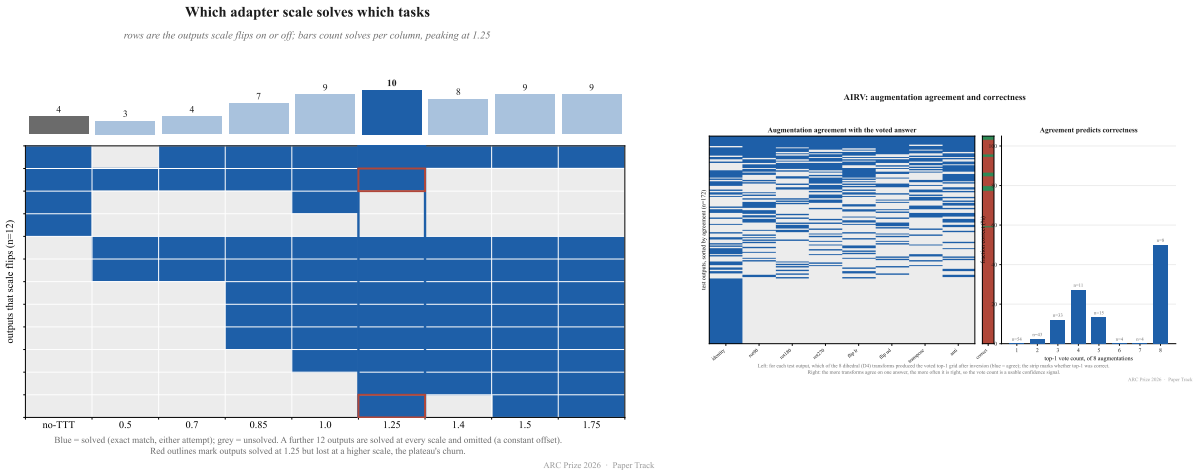


Figure 6: The left panel shows which tasks TTT solves at each adapter scale, with most gains concentrated at scale 1.25. The right panel shows AIRV vote agreement (Eq. (2)) predicting correctness, where full eight-way consensus is right far more often than split votes.

5 Discussion

5.1 Four lessons that generalize

First, on a single GPU the inference engine is the dominant design choice. Quantization and sampling mattered far less than the 13× we recovered by moving decode off the eager path. Second, the real obstacle for TTT is the *serving bridge*. Training needs a gradient framework, fast generation needs an optimized engine, and per-task TTT produces a fresh adapter the engine must serve. We show this bridge is buildable on one card, and that once it exists the open question is not plumbing but *regularization*, because the small per-task training set overfits easily. Third, a 4B model is bimodal on these tasks. It solves or it misses with little partial credit, so the refinement family that defines the current frontier needs an external partial-credit signal to work at this scale. A method that only re-ranks the model's own outputs stalls (Section 5.3).

Fourth, public-eval gains must be re-validated in the exact deployed condition, because they do not transfer on their own. The level drops, the no-TTT system halves from 12.79% public to 6.53% hidden. More telling, the deltas reverse. The three candidate-side levers that lifted or tied the public evaluation, colour permutation with multi-scale voting, more-candidates sampling, and the plausibility filter, each regressed the hidden TTT score when submitted (Table 1). Two mechanisms compound. A lever validated on the no-TTT base meets a different system once the adapter has sharpened the vote, and a lever that measures neutral on one task distribution still carries tail risk on another. The operational rule this bought is that a lever counts as validated only in the exact submission condition, and a lever that measures neutral is never shipped.

5.2 Accuracy per GPU-hour

We argue the right framing for compute-poor ARC work is **accuracy per GPU-hour** (Figure 7), and the map of walls reported here (the decode ceiling, the VRAM limit on large-grid TTT, and the overfitting of naive adapters, all in Figure 7) is a first step toward it. The findings tell a practitioner with similar hardware exactly which problems to solve first. Fix the engine before touching anything else, budget VRAM by largest-train-pair length, and regularize the per-task adapter rather than training it harder.

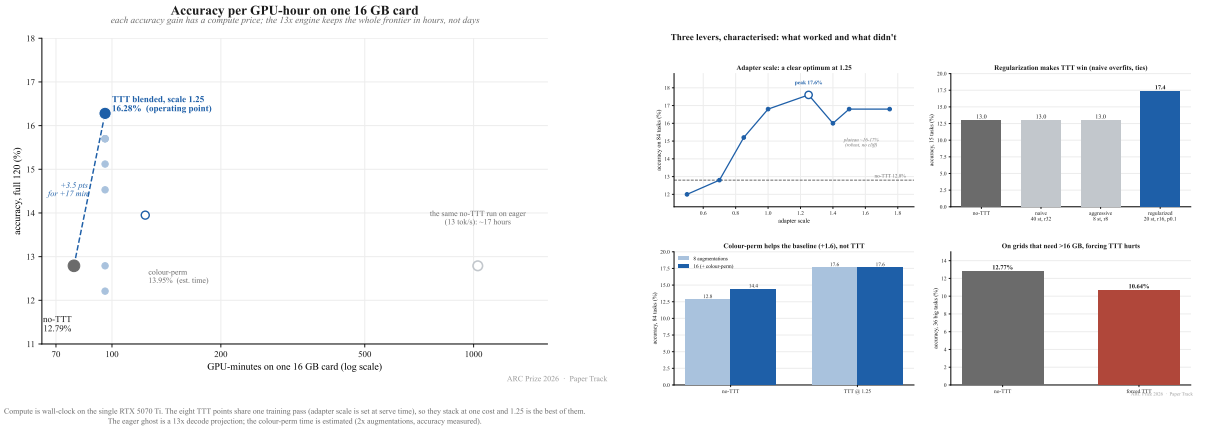


Figure 7: The left panel shows accuracy per GPU-hour on one card, where the TTT blend at 16.28% sits on the Pareto front (compute is measured wall-clock, and any eager "ghost" points are 13× projections, labelled in the source). The right panel shows what worked and what did not. The engine swap and regularized TTT helped, while naive TTT, colour-perm stacking, and big-grid TTT did not.

5.3 Limitations and ongoing work

Our absolute accuracy is below NVARC's 24.0% winning result, by design. That score was set offline on four L4 GPUs, not a cluster, by a more developed recipe on the same Qwen3-4B base lineage, so the gap reflects recipe sophistication (synthetic data, ensembling, TTT scale) more than hardware. We study the one-card inference-plus-TTT recipe rather than maximize a system. The public-to-hidden gap is a finding, not a caveat (Section 5.1). Levels drop, the no-TTT system carries 12.79% public to 6.53% hidden while the TTT blend carries 16.28% to 13.61%, and deltas reverse, with every public-validated candidate-side lever regressing when submitted (Table 1). The public ARC-AGI-2 eval is the set every open recipe is tuned against, ours included, so the leaner the recipe the more of its public accuracy is public-specific. Part of the v1 drop may also be per-task failures in the scored rerun, which the later submissions instrument with a per-task log, and the TTT system's smaller gap is consistent with both readings. The 2025 analysis names verifier-guided selection and a refinement loop (check each candidate against the train pairs, then re-prompt on the discrepancy) as the central advance, so we built and tested it. It does not lift here. A passive form (weighting the augmentation vote by leave-one-out train fidelity) and an active form (verification-gated re-sampling on the tasks the base is unsure of) each match the plain no-TTT vote exactly on the eval slices we ran. The reasons are structural. The verifier is silent exactly where it is needed, since on a task the base misses it reproduces none of the held-out train pairs, so there is nothing

to select between or correct toward. Our 16-token model also cannot read a natural-language critique, so the canonical re-prompt-with-the-discrepancy step is not even expressible. We also probed whether a one-time continued LoRA fine-tune of the base on official ARC-AGI-2 training data and RE-ARC [6] generations yields a better static model. It did not. The fine-tuned base scores 11.63% no-TTT against the stock base's 12.79%, so we ship the stock base. The NVARC base is already well-adapted to this grid format, and continued fine-tuning on this public data did not transfer. That is a useful negative result for the compute-poor practitioner. More public data is not a free lever once the base is task-tuned.

Building on that null result, we tested an expert-iteration self-training loop. We solve the official training tasks with the current model, distil the base on a curriculum of its own verified-correct solutions, and iterate. Exact-match grids impose a sharp constraint. A verified-correct solution on a training task simply is its ground-truth output, so a naive self-training corpus coincides with the data the continued fine-tune already used. We hoped to escape this through the curriculum, distilling only the near-miss tasks the base reaches but does not solve robustly, but a 200-task harvest shows that set is nearly empty. The base self-solves 194 of 200 training tasks under some augmentation (97%) and 191 by the robust vote (95.5%), leaving only three near-miss tasks. The base is saturated on its own training distribution, so self-training has no signal to add. The training tasks split into already-solved (nothing to learn) and unsolved (no correct trace), with almost nothing between. This is a second, sharper negative result on weight refinement, and it explains the fine-tune failure directly. On one GPU, with a base already tuned to the grid format, neither continued fine-tuning nor self-training moves ARC-AGI-2, because the public training data carries no signal the base has not already absorbed.

These three null results, the fine-tune, self-training, and the refinement loop, share one cause. The base is bimodal on ARC. It solves a task robustly or misses it entirely, with almost no partial-understanding middle (of the training tasks it harvests, 95.5% are solved robustly and only 1.5% are near-misses). Every method that refines, re-ranks, or distils the model's own understanding therefore has nothing to grip on the tasks it misses, and per-task TTT, the one lever that helped, is the exception that proves the rule. It does not refine partial understanding. Instead it manufactures new capability for the task. The implication for compute-poor ARC is concrete. The next lever must supply an external, continuously-gradable signal that is valid even where the base is lost, a program checked against the train pairs being the cleanest, rather than another pass over the model's own grid outputs.

We built that arm and measured it. A 1.5B model trained to emit programs in an executable DSL, sampled and kept only when a program reproduces every train pair, solved zero of the 101 ARC-2 evaluation tasks our transductive system misses. It generalized weakly on the easier ARC-1, about one held-out task in nine, but on ARC-2 the DSL cannot express the new compositional concepts, and the tasks the base misses are also the largest grids where the small proposer exhausts the card before it can search. The external-signal lever is therefore real but bounded by model scale on one GPU, the same wall the refinement family hits from the other side. The lever exists but needs a larger model to grip, which is exactly the move the full contest hardware allows (Section 5.4).

Finally, the significance of the TTT lift is directional rather than conclusive (Appendix B). A full-eval paired run is the obvious next measurement.

5.4 Scaling to the full contest hardware

Everything above holds a single 16 GB card fixed, but the offline contest gives four L4 GPUs, 96 GB together, and that hardware is where the real climb happens. As of late June 2026 the live public leaderboard is led by one entry at 50.6%, with the chasing pack between 32 and 35%, and every one of those runs uses the same offline no-internet sandbox our system does. The higher numbers often quoted for ARC-AGI-2, a 54% Gemini refinement harness and frontier scores above 70%, come from a separate public track that allows internet and external compute, so they are not the bar we measure against [3]. The offline bar is the leaderboard above. The field runs test-time adaptation and program search on the full 96 GB, and our 13.61% leaderboard score sits below it on one 16 GB card. None of our negative results argues against that climb, but none of them tests it either, so we state the scaling case as a falsifiable hypothesis for future work rather than as a claim. Every wall we mapped belongs to a 4B model on a single card, and the sharpest is bimodality. The model solves a task robustly or misses it entirely, which is the exact reason refinement and verifier-guided selection found nothing to grip. The hypothesis is that bimodality is a property of this model's scale and training rather than of the task. The falsifiable prediction is that a more capable base run in the 96 GB produces partially-correct candidates on tasks it does not solve outright, a measurable non-empty near-miss band where ours is 1.5%, and that this in turn reactivates the refinement and program-search levers that stalled here. Two of our measurements bound the hypothesis from below. A second small transductive model buys almost nothing, since our fine-tuned variant solves only two tasks the stock 4B misses, and the released TRM model, the strongest small grid model available, scores far below our pipeline while needing more compute than one card gives it, so any diversity that pays must come from capability rather than from a second small vote. One published result warns against reading the hypothesis as a bare parameter-count claim. A $\sim$200M encoder-decoder system with test-time training and white-box filtering reports 27% on the ARC-AGI-2 semi-private set, ahead of the 4B-based 2025 winner [7], so method quality can dominate size, and our own four failed elaborations of the 4B's candidate machinery (Table 1) sharpen the same point from below. We read the 16 GB result as the place where this small-model recipe runs out. The prediction above is the test of what replaces it, and we leave it as stated, a hypothesis with a measurement that would falsify it, rather than as a promise.

5.5 A routed dual-arm system

Section 5.3 argued that the next lever must supply an external, continuously-gradable signal that is valid where the base is lost, and Section 5.4 offered one escape, a larger non-bimodal base, as a falsifiable hypothesis. We are building a second escape that stays on the single-card efficiency budget rather than betting on scale, and it follows directly from two measurements already in this paper. The AIRV vote agreement predicts correctness (Figure 6), so the grid model reports for free which tasks it has solved and which it has no grip on. And the base is bimodal, so on a missed task there is nothing to refine, only something to replace.

The system routes on the first fact and fills the gap with a method that does not share the base's blind spot. High-consensus tasks are banked with the grid model's own answer, which is most of the current score at a fraction of the compute. Low-consensus tasks, where the transductive model scores near zero, are routed to an induction arm, a code model that writes plain-Python grid transforms which we execute

against the train pairs. A program that reproduces every train pair is verified on observable data in a way no vote can be, so its output becomes the first attempt and the grid model's guess falls back to the second. The two arms are complementary by construction rather than by hope, induction expresses the precise compositional rules transduction misses.

The design is deliberately do-no-harm. A banked task is byte-identical to the transductive submission, and a program is trusted only when it reproduces every train pair, so the arm can only add coverage on tasks the base was already missing. This is the property the five candidate-side levers lacked. Each of them rewrote the vote on every task including the ones already solved, and each regressed (Table 1). The one loss channel, a program that fits every train pair yet is wrong on the test and displaces a correct second attempt, is measured on the local eval before any submission. The arm also inverts the engine economy of Section 3.2. Verifying a program by running it is cheap, and where several programs survive we rank their outputs under the grid model's per-view likelihood, which is prefill-priced and about thirty times cheaper than the decode that produced them, so the system generates less and checks more. The whole arm runs inside the idle two-thirds of the offline budget the transductive submission leaves unspent.

This is the same idea whose cheap form failed in Section 5.3, and it is built to answer why. That probe used a 1.5B model emitting a fixed ARC-1 domain-specific language and solved none of the 101 missed tasks. The arm here uses a 14B code model emitting unrestricted Python, which addresses both named causes at once, model capacity and an expressive target language.

Status. The first probe is a disjoint-coverage run on the single card. A 14B code model (Qwen2.5-Coder) samples sixteen programs per task at temperature 0.8, on exactly the 101 evaluation tasks the transductive system misses, and a program is kept only if it reproduces every train pair. It solved none of them. More telling than the zero, across roughly 1600 programs only one task saw a program reproduce even a single train pair, so the model is not close anywhere. The programs are not the failure mode the arc-dsl proposer was. They are fluent and sensible. On a task whose output size changes with every example the model wrote a clean routine that extracts and sorts rows, which crashed because its fixed-shape hypothesis cannot produce a variable-size output, and on a task with a narrower output it wrote correct code for the wrong rule and returned the input width. The 1.5B DSL proposer failed because it could not express the concepts. This larger model expresses them freely and still cannot infer the rule from a few examples. The bottleneck is inference, not expression, and a general coder does not close it zero-shot on one card. Priming the coder with three verified worked-example solvers, one of them a variable-size crop aimed squarely at that failure, did not change the outcome. It solved none of the 101 and lifted the partial signal only from one task to two, with nothing reaching a near-miss. Two forms of a strong single-card coder, cold and primed, both come back empty, which is itself the result. The bottleneck is rule inference on the hardest tasks, and neither sampling more programs nor showing worked examples closes it at 14B on one card. A same-size reasoning model, which works through the transformation at length before it writes code, does not change the verdict either. On a bounded run over twelve of the missed tasks it solved none and reproduced no train pair, at roughly four minutes of reasoning per sample. Three model classes, a small restricted proposer, a strong instruct coder cold and primed, and a reasoning model, all fail to induct these hardest tasks on a single card, and the near-miss band stays empty throughout. We read this as a decisive single-card result rather than a tuning gap. The router is sound, vote agreement does report which tasks the grid model has solved, but the arm it would route to cannot grip the tasks the grid model misses

without more compute than one card provides. That is the same wall Section 5.4 predicts, now measured from the induction side, and it is where the second arm's value waits on the full contest hardware.

6 Conclusion

The 2025 winning recipe survives a single 16 GB consumer GPU, but only after the inference engine is fixed. That one change is worth more than every other knob, and it is what makes per-task TTT tractable through an adapter-serving bridge. The honest result is a 12.79% no-TTT baseline and a 16.28% regularized-TTT blend on the public evaluation, 13.61% on the hidden leaderboard, with the gains bounded by adapter overfitting and a hard VRAM wall on large grids. The sharpest general lesson is that public-eval gains must be re-validated in the deployed condition, since every public-validated candidate lever we submitted regressed on the hidden set. Those bounds belong to one small card, not to the task. The contest gives four L4 GPUs and 96 GB, where the offline frontier already passes 50%, and the verifier, the TTT bridge, and the selection rule we built are the pieces we would carry onto that hardware, with the scaling case stated as a falsifiable hypothesis rather than a promise (Section 5.4). The 16 GB number is where the climb starts, not where it ends. The full system is open source, the GGUF conversion recipe for the custom-vocabulary model, the AIRV pipeline, the per-task TTT bridge and sweep harness, and a development log recording every measurement and dead end behind these numbers.

A Reproducibility

Environment. A single NVIDIA RTX 5070 Ti (16 GB, Blackwell, $\approx$ 896 GB/s), native Windows 11. For inference, a prebuilt llama.cpp CUDA 12.4 `llama-server`, run on Blackwell through PTX JIT. For training, PyTorch with a LoRA via PEFT, bf16 base, gradient checkpointing. The two frameworks are never co-resident in VRAM (Section 3.4).

The two GGUF converter patches. The NVARC model has a 16-token *WordLevel* tokenizer, which trips the stock converter twice. (1) *Unknown tokenizer hash:* the loader does not recognize the tokenizer's hash and refuses to guess a pre-tokenizer. Its pre-tokenizer is in fact the `qwen2` regex, so we map the hash to `qwen2` explicitly. (2) *Empty merge list:* a WordLevel tokenizer has no byte-pair merges, but the loader requires a non-empty list, so we add two throwaway merges among the role tokens, which never fire on grid text. Both patches are in the repository's conversion script and are applied automatically by the Kaggle prep notebook.

Serving flags. `llama-server` is launched with full GPU offload (`-ngl 99`), a 16,384-token context, a single slot (`-np 1`), and the prompt cache disabled (`-no-cache-prompt`). Its own logs are redirected to a file so they never flood the notebook. Decoding is greedy (temperature 0) with `n_predict` sized to the target grid.

Hyperparameters. AIRV uses the eight D4 augmentations and votes to two attempts. Per-task TTT (regularized): rank 16 to 32, dropout 0.05 to 0.1, 8 to 20 steps, AdamW, a fresh random augmentation per step, leave-one-out target, context capped at $\approx$ 1,600 tokens, applied at adapter scale 1.25 at inference.

Exact values per experiment are in the development log.

B Extended results

Per-scale accuracy (84 trainable tasks). Table 4 gives the full adapter-scale sweep behind Figures 5 and 6. Accuracy rises to a peak at scale 1.25 and then declines, the signature of an adapter applied too strongly.

Table 4: No-TTT vs. TTT accuracy on the 84 trainable tasks as a function of adapter scale.

adapter scale	no-TTT	0.50	0.70	0.85	1.00	**1.25**	1.40	1.50	1.75
accuracy (%)	12.8	12.0	12.8	15.2	16.8	**17.6**	16.0	16.8	16.8

Paired significance. On the 84 trainable tasks, TTT and no-TTT disagree on seven test outputs. TTT is correct on six that no-TTT misses and incorrect on one that no-TTT gets. An exact two-sided McNemar test on a $6/1$ split gives $p \approx 0.13$, so the lift is not individually significant. We report it as directional, corroborated by the configuration sweep ($+1$) and the representativeness check (no-TTT scores 12.8% on the subset versus 12.79% on the full set).

Colour permutation. Adding a random colour permutation to the augmentation set lifts the no-TTT baseline from 12.8% to 14.4% on the 84 tasks, but provides no further gain on top of the per-task adapter, which has already learned the task's colour mapping, so the two do not stack.

Deep test-time training and the training context. The configuration sweep in Table 2 conflated two variables. Its deeper cell ran 40 steps at rank 32 with no dropout and its shallow cell 20 steps at rank 16 with dropout, so more steps was only ever measured with the regularizer removed. Table 5 separates them on all 84 trainable tasks, trained at the 2000-token context the submitted system uses. Depth alone does nothing. Sixty steps at rank 16 with dropout ties the twenty-step baseline at 24 of 125. Adding capacity while keeping the regularizer on is what moves the number. Sixty steps at rank 32 reaches 27 of 125, the best trainable-task result we have measured, and it prefers the same 1.25 inference scale. The honest caveat is variance. Two seeds of the baseline configuration score 24 and 22, so a single output sits inside the noise, and the deep result rests on one seed. A second seed is the confirmation and its verdict is in the status below. The same sweep isolates the training context as a real knob. The baseline trained at 1600 tokens scores 16.0% against 19.2% at 2000, a three-point gap wider than the seed noise. This reconciles a measurement detail. The trainable-task figure in Section 4.3 and the blend it feeds were measured at the 1600-token context of the sweep harness, while every submission from v2 on trains at 2000, so those reported numbers describe a lighter configuration than the leaderboard one and are a conservative floor for it.

Table 5: Deep-regularized TTT sweep on the 84 trainable tasks (125 test outputs). Dropout is 0.1 and the training context is 2000 tokens except where noted.

configuration	steps	rank	scale	context	correct/125 (%)
baseline (v2.1 config)	20	16	1.25	2000	24 (19.2)
baseline, seed 1	20	16	1.25	2000	22 (17.6)
deep	60	16	1.25	2000	24 (19.2)
deep	60	16	1.00	2000	23 (18.4)
deep + rank	60	32	1.25	2000	**27 (21.6)**
deep + rank	60	32	1.00	2000	25 (20.0)
baseline, short context	20	16	1.25	1600	20 (16.0)

Status. Confirmed. A second seed of the rank-32 deep configuration scores 26 of 125 and the first scored 27, while the two baseline seeds score 24 and 22, so the two configurations separate with no overlap and the roughly three-output lift is a real effect rather than a seed draw. This is the first configuration we have measured that beats the shipped operating point on the mechanism that works, per-task training itself, rather than on the candidate-and-voting side where five submissions regressed. Whether it clears the 13.61% hidden score is a submission question, and its transfer prior is better than those five because it improves the adapter rather than diluting the vote, but the public-to-hidden gap of Section 5.1 forbids assuming it. The next submission carries this configuration and we report the hidden number here.

C Engineering log of walls and dead ends

We record the failures explicitly, because on this hardware they are most of the work.

- **Eager decode is overhead-bound, and the usual fix is unavailable.** Hugging Face eager decode runs at ≈ 13 tok/s, about 10% of the bandwidth roofline. `torch.compile` would close the gap, but it needs Triton, which has no native-Windows backend. The resolution was to change engines, not to optimize the eager path.
- **The server prompt cache destabilizes unique-prompt workloads.** Enabled by default, it reserves ≈ 680 MiB per unique prompt. Every ARC prompt is unique, so it never reuses and instead destabilizes sustained generation. Disabling it and running a single supervised stream fixed the stalls.
- **4-bit QLoRA is broken on Blackwell + Windows.** bitsandbytes fails to load on this platform, so the planned 4-bit continued fine-tune was infeasible. Because per-task TTT already runs in bf16 inside 16 GB, we use a bf16 LoRA throughout and drop the 4-bit path entirely.
- **PyTorch decode can turn quadratic after a training step.** Generation in the gradient framework is not only slow but intermittently degrades after an optimizer step. This is precisely why the serving bridge exists, since we never generate in PyTorch in the scored pipeline.

- **Hugging Face eager stalls offline validation, the same engine lesson again.** The rig validation scripts that drove generation through eager had no per-task timeout, so one big-grid task could hang a whole multi-hour run with no way to resume. Moving validation onto the same `llama-server` path used for scoring, with a hard per-request cap, a heartbeat file, and per-task checkpoints, removed the stalls and made the runs resumable. The engine was the bottleneck for validation just as for inference.
- **The VRAM wall on large grids is real.** The largest-train-pair token length, not the test grid size, is the binding constraint. Tasks above the operational cutoff cannot train in 16 GB. Forcing TTT onto them by truncation did not recover accuracy, so they are excluded and served no-TTT.
- **Naive TTT ties the baseline.** The standard adapter configuration drives training loss to 0.000 and generalizes no better than no-TTT. Only regularization (fewer steps, lower rank, dropout) recovers a lift.

D Prompt and serialization format

The model's 16-token vocabulary is given in Table 6. A grid is serialized row by row, one token per cell (the token id equals the colour), rows separated by the newline token. Prompts are assembled at the id level as a ChatML few-shot sequence of train pairs followed by the test input, ending at the `ASSISTANT` turn for the model to complete. Because the role and special tokens have no textual spelling, the whole sequence is built from ids directly, never by tokenizing a string, which is what guarantees parity between training and inference.

Table 6: The 16-token grid vocabulary.

token ids	meaning
0 to 9	the ten ARC grid colours (id = colour)
10	newline / row separator
11	`USER` role marker
12	`ASSISTANT` role marker
13	`PAD`
14	`IM_START`
15	`IM_END`

E Data and model provenance

The base model is Qwen3-4B [5] (Apache-2.0), supervised-fine-tuned on ARC grids by NVARC [4]. Any derived weights we redistribute are released under Apache-2.0 with the required `NOTICE`. Evaluation and training tasks are the official ARC-AGI-2 corpus [2] (Apache-2.0). The fine-tune probe additionally uses RE-ARC [6] (MIT). Our own code and weights are released under MIT-0. We never train on the

evaluation tasks. Full license texts and attribution are in the repository's `THIRD_PARTY_LICENSES.md` and `NOTICE`.

BibTeX.

```
@article{chollet2019measure,
  title={On the Measure of Intelligence},
  author={Chollet, Francois}, journal={arXiv:1911.01547}, year={2019}}
@techreport{arcagi2,
  title={ARC-AGI-2 Technical Report},
  institution={ARC Prize Foundation}, year={2025}}
@techreport{arcprize2025,
  title={ARC Prize 2025: Technical Report},
  institution={ARC Prize Foundation}, note={arXiv:2601.10904}, year={2026}}
@misc{qwen3, title={Qwen3 Technical Report}, author={Qwen Team}, year={2025}}
@misc{nvarc2025,
  title={NVARC: ARC Prize 2025 (ARC-AGI-2) 1st-place solution},
  author={{NVIDIA Kaggle Grandmasters}}, year={2025}}
@article{hodel2024rearc,
  title={Addressing the Abstraction and Reasoning Corpus via
         Procedural Example Generation},
  author={Hodel, Michael}, journal={arXiv:2404.07353}, year={2024}}
```

References

[1] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[2] ARC Prize Foundation. ARC-AGI-2 Technical Report. 2025. `https://arcprize.org`.

[3] ARC Prize Foundation. ARC Prize 2025: Technical Report. arXiv:2601.10904, 2026.

[4] NVIDIA Kaggle Grandmasters (NVARC). ARC Prize 2025 (ARC-AGI-2) 1st-place solution. 2025.

[5] Qwen Team. Qwen3 Technical Report. 2025.

[6] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024.

[7] W. Lemes de Oliveira et al. ARC-AGI-2 Technical Report (LongT5-based system with test-time training). arXiv:2603.06590, 2026.